



INSTITUTO TECNOLÓGICO DE LAS AMÉRICAS

Integrantes:

Gilenni Moreno 2024-1705

Oswaldo Cuevas 2024-2602

Michael Moronta 2024-1696

Gabriel Paulino 2023-1083

Materia:

Programación Paralela (C2-2026)

Profesor:

Erick Leonardo Perez Veloz

Proyecto final:

FraudShield

Sistema de Detección de Fraude Bancario mediante Programación Paralela

Santo Domingo, República Dominicana

Agosto 2026

<https://github.com/fairybugs/FraudShield>

Contenido

Capítulo 1. Introducción	4
1.1 Presentación general del proyecto	4
1.2 Justificación	5
1.3 Objetivos	6
Objetivo general	6
Objetivos específicos	6
Capítulo 2. Presentación general del proyecto.....	7
2.1 Contexto del problema.....	7
2.2 Aplicación del problema en un escenario real	7
2.3 Importancia del paralelismo en la solución	8
Capítulo 3. Cumplimiento de los Requisitos del Proyecto	9
3.1 Ejecución simultánea de múltiples tareas	9
3.2 Necesidad de compartir datos entre tareas.....	9
3.3 Exploración de diferentes estrategias de paralelización	10
3.4 Escalabilidad con más recursos	10
3.5 Métricas de evaluación del rendimiento	10
3.6 Aplicación a un problema del mundo real	11
Capítulo 4. Diseño de la Solución.....	11
4.1 Arquitectura general del sistema.....	11
4.2 Diagrama de tareas paralelas	13
4.3 Estrategia de paralelización utilizada.....	14
4.4 Herramientas y tecnologías empleadas	15
Capítulo 5. Implementación Técnica	15
5.1 Descripción de la estructura del proyecto.....	15
5.2 Explicación del código clave	16
5.3 Uso de mecanismos de sincronización	17
5.4 Justificación técnica de las decisiones tomadas.....	17
Capítulo 6. Evaluación de Desempeño	18
6.1 Comparativa entre la ejecución secuencial y paralela	18
6.2 Métricas de rendimiento	18

6.3 Gráficas o tablas con resultados.....	19
6.4 Análisis de cuellos de botella y limitaciones	19
Capítulo 7. Trabajo en Equipo	20
7.1 Descripción del reparto de tareas	20
7.2 Herramientas utilizadas para la coordinación	21
Capítulo 8. Conclusiones	21
8.1 Principales aprendizajes técnicos.....	21
8.2 Retos enfrentados y superados.....	22
8.3 Posibles mejoras o líneas futuras	22
Capítulo 9. Referencias	23
Capítulo 10. Anexos.....	23
10.1 Manual de ejecución del sistema	23
10.2 Capturas adicionales	24
10.3 Repositorio del proyecto	27

Capítulo 1. Introducción

1.1 Presentación general del proyecto

La transformación digital del sector financiero ha permitido que las transacciones bancarias sean cada vez más rápidas, accesibles y eficientes. Sin embargo, este crecimiento también ha incrementado la cantidad de intentos de fraude, convirtiéndose en uno de los principales desafíos para las instituciones financieras. Cada día se procesan millones de operaciones electrónicas y, dentro de ese gran volumen de información, pueden existir transacciones con características anómalas que representen un riesgo para los clientes y para las entidades bancarias.

Ante esta realidad, resulta fundamental contar con sistemas capaces de analizar grandes cantidades de datos en el menor tiempo posible, identificando patrones sospechosos antes de que una operación sea completada. No obstante, realizar este tipo de análisis de manera secuencial puede provocar tiempos de respuesta elevados cuando el número de transacciones aumenta considerablemente, afectando la capacidad de reacción del sistema.

Con el propósito de abordar esta problemática, el presente proyecto desarrolla **FraudShield**, un sistema inteligente para la detección de fraude bancario utilizando técnicas de programación paralela en C#. La aplicación procesa un conjunto masivo de transacciones financieras y analiza simultáneamente distintos indicadores de riesgo, como el monto de la operación, el país de origen, el horario en que se realiza, la frecuencia de transacciones y el tipo de comercio involucrado. Cada uno de estos criterios es evaluado mediante tareas independientes que trabajan de forma concurrente sobre la misma información.

La estrategia de paralelización seleccionada para este proyecto es la paralelización especulativa, debido a que permite ejecutar varios análisis al mismo tiempo sin esperar el resultado de los demás. Una vez finalizadas todas las evaluaciones, el sistema integra los resultados obtenidos para cada analizador asigna un puntaje de riesgo a la transacción. Posteriormente, el sistema acumula dichos puntajes y compara el riesgo total con un umbral predefinido para decidir si la transacción debe aprobarse o cancelarse.

Además del desarrollo funcional del sistema, este proyecto busca demostrar las ventajas del procesamiento paralelo frente a la ejecución secuencial mediante la comparación de tiempos de ejecución, eficiencia y capacidad de escalabilidad al incrementar el volumen de datos analizados. Para ello se trabajará con un archivo que contendrá hasta un millón de transacciones simuladas con características similares a situaciones reales, permitiendo evaluar el comportamiento del sistema bajo una carga considerable de información.

Finalmente, este proyecto integra conceptos de programación paralela, sincronización de tareas, medición del rendimiento y organización colaborativa mediante herramientas de control de versiones, con el objetivo de ofrecer una solución que represente un escenario cercano al ámbito profesional del desarrollo de software.

1.2 Justificación

El fraude bancario representa una de las principales amenazas para las instituciones financieras debido al impacto económico y operativo que genera. Las entidades bancarias procesan diariamente millones de transacciones provenientes de diferentes canales digitales, por lo que identificar operaciones sospechosas de forma rápida constituye un requisito indispensable para reducir pérdidas y proteger la información de los clientes.

El tema fue seleccionado porque permite abordar un problema real mediante técnicas de programación paralela, cumpliendo con los objetivos establecidos para el proyecto final de la asignatura. La detección de fraude requiere analizar múltiples criterios sobre una misma transacción, lo que hace posible ejecutar diferentes tareas de manera simultánea y posteriormente combinar sus resultados para obtener una decisión final. Este comportamiento se adapta de forma natural a la estrategia de paralelización especulativa, ya que cada análisis puede realizarse de manera independiente mientras comparte la información necesaria para calcular el nivel de riesgo.

Otro aspecto que motivó la elección de este proyecto es la posibilidad de trabajar con un gran volumen de datos. El sistema será capaz de generar y procesar hasta un millón de transacciones simuladas con características similares a situaciones reales, permitiendo evaluar cómo influye el paralelismo en el rendimiento de la aplicación y comparar los resultados obtenidos mediante ejecución secuencial y paralela.

Además de cumplir con los objetivos académicos de la asignatura, este proyecto permite fortalecer conocimientos relacionados con programación concurrente, sincronización de datos, medición del desempeño, organización colaborativa mediante Git y desarrollo de aplicaciones en C#, habilidades ampliamente utilizadas en proyectos de software del ámbito profesional.

1.3 Objetivos

Objetivo general

Desarrollar un sistema inteligente de detección de fraude bancario utilizando programación paralela en C# mediante la estrategia de paralelización especulativa, capaz de analizar un gran volumen de transacciones financieras, identificar operaciones con alto riesgo de fraude y comparar el rendimiento entre una ejecución secuencial y una ejecución paralela.

Objetivos específicos

- Diseñar un generador de datos capaz de crear hasta un millón de transacciones con características similares a situaciones reales.
- Implementar diferentes analizadores especializados que evalúen de manera independiente criterios de riesgo como el monto, el país de origen, el horario de la transacción, la frecuencia de operaciones y el tipo de comercio.
- Aplicar la estrategia de paralelización especulativa para ejecutar simultáneamente todos los análisis asociados a cada transacción.
- Comparar el tiempo de ejecución entre una implementación secuencial y una implementación paralela mediante métricas de rendimiento.
- Analizar la eficiencia y la escalabilidad del sistema al incrementar progresivamente la cantidad de transacciones procesadas.
- Integrar herramientas de control de versiones y trabajo colaborativo para facilitar el desarrollo organizado del proyecto.

Capítulo 2. Presentación general del proyecto

2.1 Contexto del problema

El crecimiento de las plataformas digitales y el aumento de las transacciones electrónicas han transformado la forma en que las personas realizan operaciones financieras. Actualmente, es posible efectuar pagos, transferencias y compras en línea en cuestión de segundos desde cualquier parte del mundo. Aunque esta evolución ha mejorado la accesibilidad y la rapidez de los servicios bancarios, también ha incrementado las oportunidades para que personas malintencionadas intenten cometer fraudes mediante el uso indebido de tarjetas, cuentas bancarias o credenciales robadas.

Las instituciones financieras procesan diariamente millones de transacciones provenientes de diferentes canales, como aplicaciones móviles, plataformas web, cajeros automáticos y terminales de pago. Cada una de estas operaciones debe ser validada en un tiempo muy reducido para garantizar una buena experiencia al usuario. Sin embargo, dentro de ese gran volumen de información pueden existir transacciones que presentan características inusuales y que requieren una evaluación adicional antes de ser autorizadas.

Detectar este tipo de operaciones representa un desafío importante debido a que el sistema debe analizar grandes cantidades de datos sin afectar el tiempo de respuesta esperado por los clientes. A medida que aumenta el número de transacciones, también incrementa la carga de procesamiento, lo que hace necesario utilizar técnicas que permitan distribuir el trabajo de manera eficiente y aprovechar mejor los recursos del hardware disponible.

En este contexto surge la necesidad de desarrollar soluciones capaces de identificar comportamientos sospechosos mediante el análisis simultáneo de diferentes criterios de riesgo, permitiendo detectar posibles fraudes sin comprometer el rendimiento del sistema.

2.2 Aplicación del problema en un escenario real

En un banco real, una misma transacción puede ser evaluada utilizando múltiples indicadores antes de ser aprobada. Por ejemplo, el sistema puede verificar si el monto de la operación es superior al promedio habitual del cliente, si la compra fue realizada desde un país diferente al de sus transacciones anteriores, si ocurrió en un horario poco común o si el comercio donde se intenta realizar el pago presenta antecedentes de operaciones sospechosas.

Imaginemos el siguiente escenario:

Un cliente residente en República Dominicana realiza normalmente compras entre RD\$2,000 y RD\$5,000 en comercios nacionales durante el día. De forma repentina, el sistema recibe una solicitud de compra por RD\$250,000 realizada a las 3:15 de la madrugada desde un país distinto al habitual y en un comercio catalogado como de alto riesgo. Aunque cada uno de estos factores, por separado, no necesariamente confirma un fraude, la combinación de todos ellos incrementa considerablemente la probabilidad de que la transacción sea fraudulenta.

Este tipo de situaciones ocurre constantemente en los sistemas financieros modernos y requiere mecanismos capaces de analizar varios criterios de manera simultánea para tomar una decisión en el menor tiempo posible.

Con el fin de simular este comportamiento, el sistema desarrollado en este proyecto generará un conjunto masivo de transacciones con características similares a escenarios reales. El usuario podrá indicar cuántas transacciones desea generar y cuántas de ellas deberán contener condiciones fraudulentas. Esto permitirá evaluar el comportamiento del sistema bajo diferentes cargas de trabajo y comprobar la efectividad de los detectores implementados.

2.3 Importancia del paralelismo en la solución

Uno de los principales retos de este proyecto consiste en procesar un volumen elevado de información manteniendo tiempos de respuesta aceptables. Si cada transacción fuera analizada de forma completamente secuencial, el tiempo total de procesamiento aumentaría considerablemente a medida que crece la cantidad de registros, afectando la eficiencia del sistema.

Para resolver esta situación se implementará una estrategia basada en paralelización especulativa donde varios analizadores evaluarán simultáneamente la misma transacción. Cada analizador tendrá la responsabilidad de revisar un criterio específico como el monto, la ubicación geográfica, el horario, la frecuencia de operaciones o el comercio asociado. Todos estos análisis se ejecutarán de manera concurrente y, una vez finalizados, sus resultados serán integrados para calcular un porcentaje de riesgo que determinará si la transacción debe aprobarse o considerarse potencialmente fraudulenta.

La utilización de esta estrategia permite aprovechar de forma más eficiente los procesadores multinúcleo disponibles en los equipos actuales, reduciendo el tiempo necesario para analizar grandes volúmenes de información. Asimismo, facilita la comparación entre una implementación secuencial y otra paralela, permitiendo medir objetivamente el impacto del paralelismo mediante indicadores como el tiempo de ejecución, el speedup y la eficiencia.

Capítulo 3. Cumplimiento de los Requisitos del Proyecto

3.1 Ejecución simultánea de múltiples tareas

Uno de los principales objetivos del proyecto consiste en aprovechar las capacidades de procesamiento paralelo para reducir el tiempo necesario para analizar un gran volumen de transacciones financieras. Para lograrlo, cada transacción será evaluada mediante cinco analizadores independientes, cada uno especializado en un criterio diferente de detección de fraude.

Los analizadores verificarán aspectos como el monto de la transacción, el país desde donde se realiza la operación, el horario, la frecuencia de compras recientes y el comercio involucrado. Debido a que estos análisis son independientes entre sí, podrán ejecutarse de manera simultánea utilizando la Task Parallel Library (TPL) de C#.

La ejecución concurrente de estos analizadores permitirá disminuir el tiempo de procesamiento respecto a una implementación completamente secuencial, especialmente cuando el sistema procese cientos de miles o millones de registros.

3.2 Necesidad de compartir datos entre tareas

Aunque cada analizador ejecuta una función específica, todos necesitan acceder a la misma información correspondiente a la transacción que está siendo evaluada.

Por ejemplo, el analizador de monto necesita conocer el valor de la compra, mientras que el analizador de país requiere consultar el país habitual del cliente y el país desde donde se realizó la operación. De igual forma, el analizador de horario utilizará la fecha y hora registradas, y el analizador de frecuencia verificará el historial reciente del mismo cliente.

Todos estos módulos trabajarán sobre una misma instancia del objeto **Transacción**, compartiendo la información necesaria sin duplicar datos. Una vez finalizados los análisis, cada tarea devolverá su resultado para que el sistema calcule el porcentaje de riesgo total asociado a la operación.

De esta manera, el proyecto incorpora la compartición de datos entre tareas, característica fundamental dentro de los sistemas concurrentes.

3.3 Exploración de diferentes estrategias de paralelización

La estrategia principal dada para este proyecto es la **paralelización especulativa** y consideramos que se adapta de manera natural al problema planteado.

En este enfoque, todos los analizadores comienzan su ejecución al mismo tiempo sin depender de los resultados de los demás. Posteriormente, el sistema recopila los porcentajes de riesgo generados por cada uno de ellos y obtiene un resultado único que permitirá decidir si la transacción debe aprobarse o ser considerada potencialmente fraudulenta.

Además de implementar esta estrategia, el proyecto permitirá comparar su comportamiento frente a una ejecución completamente secuencial, proporcionando una visión clara del impacto que tiene el paralelismo sobre el rendimiento del sistema.

3.4 Escalabilidad con más recursos

El sistema fue diseñado para procesar diferentes cantidades de transacciones sin necesidad de modificar su arquitectura.

Durante las pruebas será posible generar conjuntos de datos de distintos tamaños, permitiendo evaluar el comportamiento de la aplicación con cargas de trabajo progresivamente mayores. El usuario podrá indicar la cantidad de transacciones que desea generar y cuántas de ellas serán fraudulentas, facilitando la realización de pruebas con diferentes escenarios.

Asimismo, al ejecutarse sobre equipos con varios núcleos de procesamiento, el sistema podrá distribuir mejor las tareas paralelas y aprovechar de forma más eficiente los recursos disponibles, demostrando la capacidad de escalar conforme aumenta la carga de trabajo.

3.5 Métricas de evaluación del rendimiento

Una parte fundamental del proyecto consiste en medir objetivamente el desempeño obtenido mediante programación paralela. Para ello se desarrollarán dos versiones del sistema: una implementación secuencial y otra paralela. Ambas procesarán exactamente el mismo conjunto de transacciones, permitiendo comparar directamente sus resultados. Estas mediciones permitirán demostrar cuantitativamente las ventajas del enfoque paralelo implementado. Entre las métricas consideradas se encuentran:

- Tiempo total de ejecución.
- Speedup obtenido mediante paralelismo.
- Eficiencia alcanzada.
- Comportamiento del sistema al incrementar la cantidad de transacciones procesadas y la cantidad de núcleos de procesamiento utilizados.

3.6 Aplicación a un problema del mundo real

El fraude bancario constituye una problemática presente en las instituciones financieras de todo el mundo. La necesidad de analizar miles o millones de operaciones en tiempos muy reducidos convierte este escenario en un excelente candidato para aplicar técnicas de programación paralela.

Aunque el sistema desarrollado corresponde a una simulación académica, su funcionamiento reproduce el comportamiento general de un mecanismo automatizado de evaluación de riesgo. La generación de transacciones con características similares a situaciones reales junto con la implementación de diferentes criterios de análisis permite aproximarse a un escenario representativo del ámbito financiero.

Como resultado, el proyecto no solo cumple con los objetivos de aprendizaje de la asignatura, sino que también demuestra una aplicación práctica de los conceptos de paralelismo sobre un problema con relevancia en el entorno profesional.

Capítulo 4. Diseño de la Solución

4.1 Arquitectura general del sistema

FraudShield fue diseñado siguiendo una arquitectura modular con el propósito de facilitar el mantenimiento, la escalabilidad y la distribución del trabajo entre los integrantes del equipo. Cada componente del sistema posee una responsabilidad específica, permitiendo reducir el acoplamiento entre módulos y favoreciendo futuras ampliaciones del proyecto.

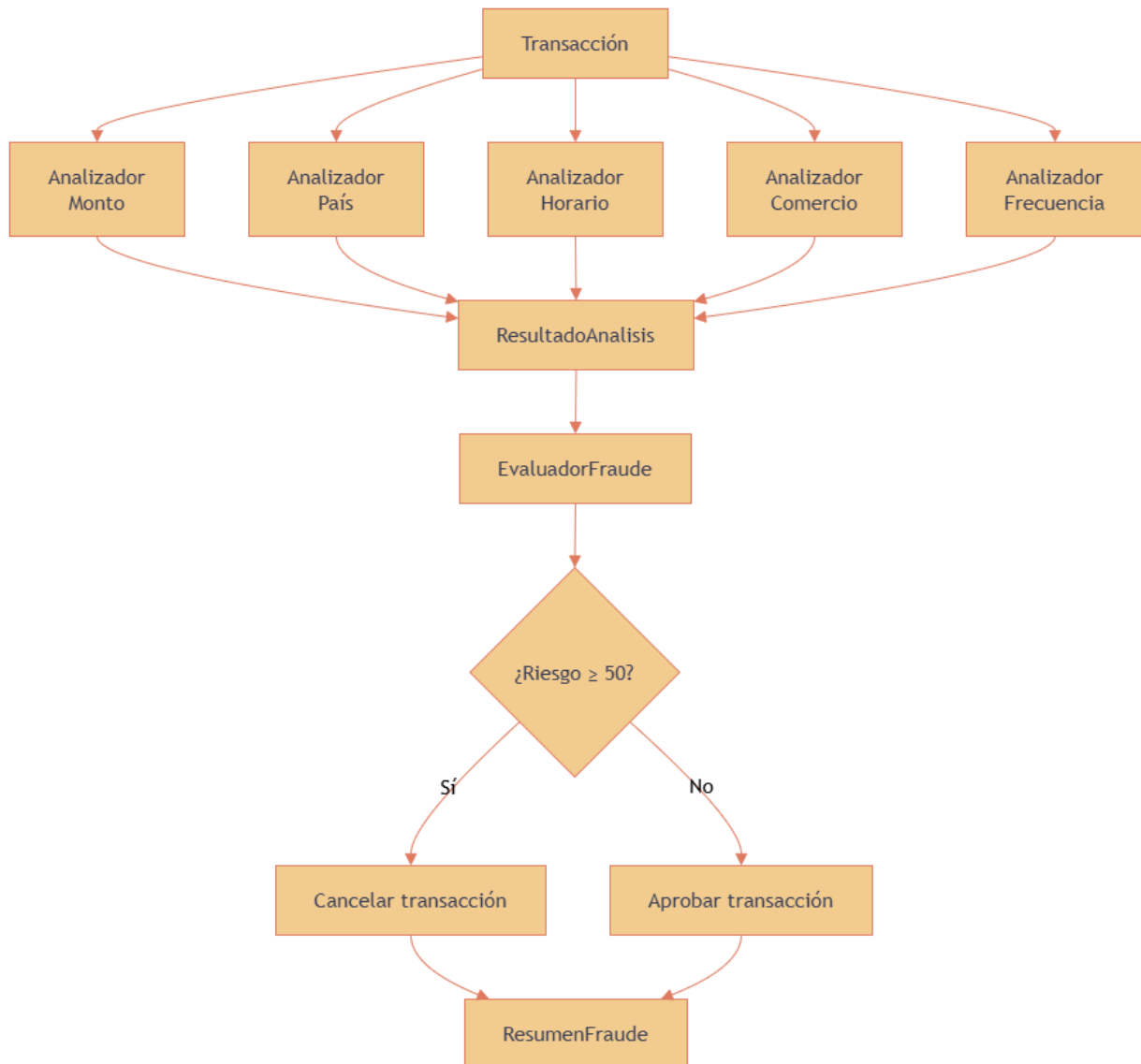
La aplicación se desarrollará como un sistema de consola utilizando el lenguaje de programación C# sobre la plataforma .NET. El procesamiento de las transacciones se dividirá en diferentes etapas, comenzando con la generación del conjunto de datos, seguida de la lectura del archivo CSV, el análisis paralelo de cada transacción, el cálculo del nivel de riesgo y la presentación de las estadísticas finales.

La arquitectura propuesta permite que cada módulo funcione de manera independiente, comunicándose únicamente mediante objetos compartidos y resultados claramente definidos. Esta organización facilita tanto el desarrollo colaborativo como la integración de nuevas funcionalidades sin afectar significativamente el resto del sistema.

La estructura general del proyecto se encuentra organizada de la siguiente manera:



4.2 Diagrama de tareas paralelas



4.3 Estrategia de paralelización utilizada

Para el desarrollo de FraudShield se implementó la estrategia de paralelización especulativa, la cual consiste en ejecutar de forma simultánea varias tareas independientes que analizan diferentes aspectos de una misma transacción bancaria. Esta estrategia fue seleccionada porque cada criterio de evaluación puede ejecutarse sin depender del resultado de los demás, permitiendo aprovechar de forma eficiente los procesadores multinúcleo disponibles.

El sistema cuenta con dos versiones del detector de fraude: una implementación secuencial y otra paralela. La versión secuencial procesa las transacciones una por una mediante un ciclo foreach, ejecutando los cinco analizadores de manera consecutiva para cada transacción. En cambio, la versión paralela utiliza la Task Parallel Library (TPL) de .NET mediante la instrucción Parallel.ForEach, permitiendo que múltiples transacciones sean procesadas al mismo tiempo.

Cada transacción es evaluada por cinco analizadores especializados:

- Analizador de monto.
- Analizador de país.
- Analizador de horario.
- Analizador de comercio.
- Analizador de frecuencia.

Cada analizador genera un objeto ResultadoAnálisis, el cual contiene el nivel de riesgo calculado, la información sobre si la condición evaluada resulta sospechosa y el motivo correspondiente. Todos estos resultados son almacenados en una colección compartida, estructura que garantiza la seguridad en el acceso concurrente por parte de múltiples hilos.

Una vez finalizado el procesamiento paralelo, los resultados obtenidos son enviados al componente EvaluadorFraude, encargado de calcular el riesgo total de cada transacción mediante la suma de los puntajes asignados por los cinco analizadores. Posteriormente, dicho riesgo es comparado con un umbral de 50 puntos. Si el riesgo acumulado es igual o superior a este valor, la transacción se clasifica como fraudulenta y se considera cancelada; en caso contrario, la transacción es aprobada.

Además, la implementación permite que el usuario seleccione la cantidad de núcleos de procesamiento que desea utilizar durante la ejecución paralela. Para ello se emplea la propiedad MaxDegreeOfParallelism de la clase ParallelOptions, lo que facilita realizar pruebas de escalabilidad y comparar el comportamiento del sistema utilizando diferentes cantidades de procesadores.

Finalmente, para evaluar el impacto del paralelismo, el proyecto incorpora un módulo de medición de rendimiento que registra el tiempo de ejecución tanto de la versión secuencial como de la versión paralela. A partir de estos datos se calculan las métricas de Speedup y Eficiencia, permitiendo cuantificar objetivamente la mejora obtenida mediante el procesamiento paralelo.

4.4 Herramientas y tecnologías empleadas

Herramienta	Uso
C#	Desarrollo del sistema
.NET	Plataforma de ejecución
Visual Studio 2026	Desarrollo del proyecto
Task Parallel Library (TPL)	Ejecución paralela
xUnit	Pruebas unitarias
Draw.io	Diagramas
CSV	Almacenamiento de transacciones

Capítulo 5. Implementación Técnica

5.1 Descripción de la estructura del proyecto

El proyecto fue desarrollado utilizando una arquitectura modular, organizando las clases en diferentes carpetas según su responsabilidad dentro del sistema. Esta estructura permitió mantener el código más ordenado y facilitar tanto el desarrollo como el mantenimiento de la aplicación.

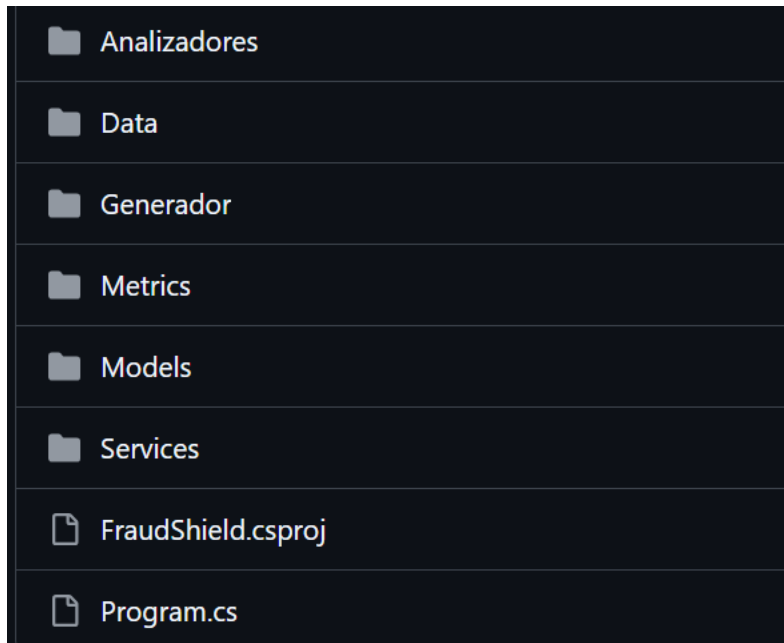
La carpeta **Models** contiene las entidades principales del sistema, como Transaccion, ResultadoAnálisis y ResultadoMedicion, las cuales representan la información utilizada durante el proceso de detección de fraude.

En la carpeta **Analizadores** se encuentran los cinco analizadores implementados: monto, país, horario, comercio y frecuencia. Cada uno evalúa un criterio específico de riesgo y devuelve un resultado independiente.

La carpeta **Services** concentra la lógica principal del sistema. Allí se implementaron el detector secuencial, el detector paralelo, el evaluador de fraude y el resumen de resultados. Estos componentes son responsables de ejecutar el análisis de las transacciones y determinar si una operación debe aprobarse o cancelarse.

La carpeta **Data** contiene las clases encargadas de generar y leer los archivos CSV utilizados durante las pruebas, mientras que en **Metrics** se encuentran las clases responsables de medir el tiempo de ejecución y calcular las métricas de rendimiento del sistema.

Finalmente, el proyecto incluye un módulo de **tests** desarrollado con xUnit, utilizado para validar el funcionamiento de los detectores implementados.



5.2 Explicación del código clave

El funcionamiento del sistema comienza cuando el usuario genera o carga un archivo CSV con las transacciones bancarias. Posteriormente, dichas transacciones son almacenadas en una lista y enviadas al detector seleccionado desde el menú principal.

La versión secuencial procesa las transacciones mediante un ciclo foreach, ejecutando los cinco analizadores para cada registro. En cambio, la versión paralela utiliza Parallel.ForEach, permitiendo que múltiples transacciones sean procesadas al mismo tiempo utilizando varios núcleos del procesador.

Cada analizador devuelve un objeto ResultadoAnálisis con el nivel de riesgo detectado y la información correspondiente. Una vez finalizado el análisis, todos los resultados son enviados a la clase EvaluadorFraude, la cual suma los puntajes obtenidos y determina si la transacción debe aprobarse o cancelarse utilizando un umbral de 50 puntos.

Finalmente, las clases MedidorTiempo y CalculadoraMetricas registran el tiempo de ejecución y calculan las métricas de Speedup y Eficiencia, mientras que ResumenFraude presenta un resumen con la cantidad de transacciones aprobadas, canceladas y las alertas de fraude detectadas.

5.3 Uso de mecanismos de sincronización

Durante la implementación de la versión paralela fue necesario utilizar mecanismos que garantizaran el acceso seguro a los datos compartidos entre los diferentes hilos de ejecución.

Para ello se utilizó la colección `ConcurrentBag<ResultadoAnálisis>`, la cual permite que varios hilos agreguen resultados simultáneamente sin provocar conflictos de concurrencia ni pérdida de información.

Asimismo, el procesamiento paralelo se implementó mediante `Parallel.ForEach` junto con la clase `ParallelOptions`, utilizando la propiedad `MaxDegreeOfParallelism` para controlar la cantidad máxima de núcleos empleados durante la ejecución. De esta forma fue posible realizar pruebas utilizando diferentes niveles de paralelismo sin modificar la lógica principal del sistema.

Gracias a estos mecanismos, el procesamiento paralelo pudo ejecutarse de forma segura y estable durante el análisis de grandes volúmenes de transacciones.

5.4 Justificación técnica de las decisiones tomadas

Durante el desarrollo del proyecto se tomaron diferentes decisiones con el objetivo de obtener una solución organizada, eficiente y fácil de mantener.

Se implementaron dos versiones del detector de fraude (secuencial y paralela) para poder comparar objetivamente el impacto del procesamiento paralelo sobre el tiempo de ejecución. Esto permitió calcular posteriormente las métricas de Speedup y Eficiencia utilizando las mismas transacciones como base de comparación.

La biblioteca **Task Parallel Library (TPL)** fue seleccionada debido a que proporciona herramientas sencillas para implementar paralelismo en aplicaciones desarrolladas con .NET. Además, permitió controlar fácilmente la cantidad de núcleos utilizados durante las pruebas mediante `MaxDegreeOfParallelism`.

También se decidió separar cada criterio de evaluación en analizadores independientes, lo que facilitó la organización del código y permitió que cada componente tuviera una única responsabilidad. Finalmente, la incorporación de clases como `EvaluadorFraude`, `ResumenFraude` y `CalculadoraMetricas` permitió mantener la lógica del sistema dividida en módulos específicos, haciendo que el proyecto fuera más limpio, reutilizable y sencillo de comprender.

Capítulo 6. Evaluación de Desempeño

6.1 Comparativa entre la ejecución secuencial y paralela

Con el objetivo de evaluar el impacto del procesamiento paralelo, se realizaron pruebas utilizando el mismo conjunto de datos en ambas versiones del sistema. Para ello se generó un archivo CSV con 1,000,000 de transacciones, de las cuales 50,000 correspondían a transacciones fraudulentas.

Las pruebas se ejecutaron utilizando el mismo equipo y las mismas condiciones para garantizar una comparación justa entre ambas implementaciones. Durante cada ejecución se registró el tiempo empleado por la versión secuencial y por la versión paralela, permitiendo posteriormente calcular las métricas de rendimiento del sistema.

Los resultados obtenidos muestran una reducción considerable en el tiempo de procesamiento al utilizar la versión paralela, demostrando que el uso de múltiples núcleos permite aprovechar mejor los recursos del procesador cuando se trabaja con grandes volúmenes de información.

6.2 Métricas de rendimiento

Durante las pruebas se calcularon tres métricas principales para evaluar el comportamiento del sistema: tiempo de ejecución, Speedup y Eficiencia.

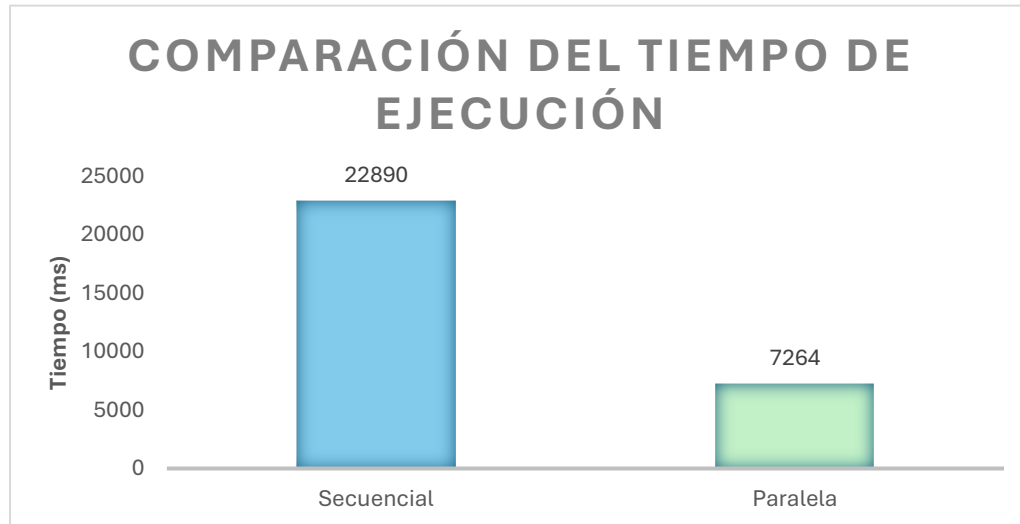
- **Tiempo de ejecución:** representa el tiempo total empleado por cada versión para procesar todas las transacciones.
- **Speedup:** indica cuántas veces la versión paralela fue más rápida respecto a la versión secuencial.
- **Eficiencia:** muestra qué tan bien fueron aprovechados los núcleos del procesador durante la ejecución paralela.

Los valores obtenidos fueron los siguientes:

Métrica	Resultado
Tiempo secuencial	22,890 ms
Tiempo paralelo	7,264 ms
Speedup	3.15
Procesadores utilizados	6
Eficiencia	52.52 %

6.3 Gráficas o tablas con resultados

Para facilitar la interpretación de los resultados, la Tabla 6.1 resume los tiempos de ejecución obtenidos durante las pruebas.



Esta gráfica permite observar claramente la disminución del tiempo de procesamiento al utilizar el algoritmo paralelo, evidenciando la mejora obtenida mediante el uso de múltiples núcleos del procesador.

6.4 Análisis de cuellos de botella y limitaciones

Aunque la implementación paralela obtuvo un mejor rendimiento, no fue posible alcanzar una eficiencia del 100 %. Esto se debe a que el procesamiento paralelo introduce costos adicionales asociados a la creación y administración de los hilos de ejecución, así como al acceso concurrente a estructuras compartidas.

Otro aspecto importante es que algunas operaciones del sistema continúan ejecutándose de forma secuencial, como la lectura inicial del archivo CSV y la generación del resumen final de resultados. Estas tareas limitan el grado máximo de aceleración que puede alcanzarse.

Asimismo, el analizador de frecuencia representa una de las operaciones más costosas del sistema, ya que necesita consultar el historial de transacciones de cada cliente para determinar la frecuencia de compras en un intervalo determinado. Este comportamiento incrementa el tiempo total de procesamiento tanto en la versión secuencial como en la paralela.

A pesar de estas limitaciones, los resultados obtenidos demuestran que el uso de programación paralela permitió reducir considerablemente el tiempo de ejecución al procesar grandes volúmenes de transacciones, cumpliendo así con el objetivo principal del proyecto.

Capítulo 7. Trabajo en Equipo

7.1 Descripción del reparto de tareas

El desarrollo del proyecto se realizó de forma colaborativa, asignando responsabilidades específicas a cada integrante del equipo. Esta distribución permitió avanzar en diferentes módulos de manera simultánea y posteriormente integrar todos los componentes en una única solución.

Integrante	Responsabilidades
Oswaldo	Desarrollo de las carpetas Models, Generador y Data.
Gilenni Moreno (Líder del proyecto)	Desarrollo de la carpeta Analizadores, elaboración de la documentación técnica, coordinación general del proyecto y apoyo en la integración de los demás módulos.
Michael Moronta	Desarrollo de la carpeta Services, incluyendo la lógica principal del sistema.
Gabriel Paulino	Desarrollo de la carpeta Metrics, encargada de la medición del rendimiento y cálculo de métricas.

Además de las responsabilidades principales, cada integrante desarrolló las pruebas unitarias correspondientes a los módulos que implementó, con el propósito de verificar el correcto funcionamiento de sus componentes antes de integrarlos al proyecto.

Como líder del proyecto, también participé en la revisión del código desarrollado por el equipo, la integración de los diferentes módulos, el apoyo en la resolución de errores y la elaboración de la documentación final del proyecto, asegurando que todos los componentes funcionaran correctamente en conjunto.

7.2 Herramientas utilizadas para la coordinación

Durante el desarrollo del proyecto se utilizaron diferentes herramientas para facilitar la organización del trabajo, el control de versiones y la documentación de los resultados obtenidos.

- **Git y GitHub:** Utilizados para el control de versiones, almacenamiento del repositorio y colaboración entre los integrantes del equipo.
- **Archivos CSV:** Utilizados para generar, almacenar y procesar las transacciones bancarias empleadas durante las pruebas del sistema.
- **Draw.io:** Utilizados para diseñar los diagramas de arquitectura, componentes y tareas paralelas incluidos en la documentación.
- **Excel:** Utilizado para elaborar tablas, generar gráficos y analizar los resultados de rendimiento obtenidos durante las pruebas.
- **Word:** Utilizado para la elaboración del informe técnico y la documentación del proyecto.
- **Microsoft Teams:** Utilizado para la comunicación, coordinación y seguimiento del trabajo entre los integrantes del equipo.

Capítulo 8. Conclusiones

8.1 Principales aprendizajes técnicos

El desarrollo de FraudShield permitió aplicar de manera práctica los conceptos estudiados en la asignatura de Programación Paralela, especialmente el uso de la Task Parallel Library para distribuir el procesamiento de grandes volúmenes de datos entre varios núcleos del procesador.

Durante la implementación también se reforzaron conocimientos relacionados con el diseño orientado a objetos, la organización modular del código, el desarrollo de pruebas unitarias y el uso de estructuras seguras para trabajar con múltiples hilos, como ConcurrentBag. Además, se aprendió a medir el rendimiento de una aplicación mediante métricas como el tiempo de ejecución, el Speedup y la Eficiencia, permitiendo comparar objetivamente el comportamiento de las versiones secuencial y paralela.

8.2 Retos enfrentados y superados

Durante el desarrollo del proyecto se presentaron diferentes desafíos tanto técnicos como organizativos. Uno de los principales retos fue adaptar la lógica del detector de fraude para que pudiera ejecutarse correctamente en un entorno paralelo sin afectar la consistencia de los resultados. Para lograrlo fue necesario utilizar estructuras seguras para el acceso concurrente a los datos y controlar la cantidad de núcleos utilizados durante la ejecución.

Otro desafío importante fue optimizar el rendimiento del sistema al procesar grandes volúmenes de transacciones. Durante las pruebas se identificó que el analizador de frecuencia representaba el mayor costo de procesamiento, ya que debía consultar el historial de transacciones para cada operación analizada. A pesar de esta limitación, la implementación paralela logró reducir significativamente el tiempo de ejecución con respecto a la versión secuencial.

En el trabajo colaborativo también surgieron retos relacionados con el uso compartido del repositorio en GitHub. Durante la integración del proyecto fue necesario resolver problemas de sincronización de cambios, actualización del repositorio local, manejo de pull requests y coordinación entre los integrantes para evitar conflictos al modificar los mismos archivos. Estos inconvenientes fueron superados mediante una mejor organización del trabajo, el uso adecuado del control de versiones y una comunicación constante entre los miembros del equipo.

8.3 Posibles mejoras o líneas futuras

Como trabajo futuro, el sistema podría incorporar nuevos criterios de detección de fraude basados en técnicas de inteligencia artificial o aprendizaje automático, permitiendo identificar patrones más complejos que los evaluados mediante reglas predefinidas.

Otra posible mejora consiste en optimizar el analizador de frecuencia utilizando estructuras de datos más eficientes o algoritmos que reduzcan el tiempo de búsqueda sobre el historial de transacciones. Asimismo, el sistema podría ampliarse para procesar datos en tiempo real, conectándose directamente con una base de datos o con servicios bancarios en línea.

Finalmente, podrían desarrollarse nuevas interfaces gráficas y paneles de visualización que permitan monitorear las alertas de fraude y las métricas de rendimiento de una manera más intuitiva para el usuario.

Capítulo 9. Referencias

Downey, A. B. (1998). Introduction to Parallel Computing. Addison-Wesley.

<https://www.oreilly.com/library/view/introduction-to-parallel/0201648652/ch02.html#ch02lev2sec10>

Hands-On Parallel Programming with C# 8 and .NET Core 3.0. (2019). Packt Publishing.

https://www.oreilly.com/videos/hands-on-parallel-programming/9781789809749/9781789809749-video1_2/

Microsoft. (s.f.). Task Parallel Library (TPL). Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/standard/parallel-programming/>

diagrams.net. (s.f.). diagrams.net. <https://app.diagrams.net/>

Capítulo 10. Anexos

10.1 Manual de ejecución del sistema

Para ejecutar correctamente el sistema FraudShield se deben seguir los siguientes pasos:

1. Abrir la solución del proyecto en Visual Studio 2026.
2. Compilar la solución para verificar que no existan errores.
3. Ejecutar la aplicación desde el menú principal.
4. Seleccionar la opción 1 para generar un archivo CSV indicando la cantidad de transacciones y la cantidad de transacciones fraudulentas.
5. Seleccionar la opción 2 para cargar el archivo CSV generado.
6. Dirigirse a la carpeta bin/Debug/net10.0 del proyecto y copiar la ruta del archivo CSV generado
7. Ejecutar la opción 3 para realizar el análisis mediante el detector secuencial.
8. Ejecutar la opción 4 para realizar el análisis mediante el detector paralelo, indicando la cantidad de núcleos a utilizar.
9. Utilizar la opción 5 para comparar los tiempos de ejecución de ambas versiones.
10. Utilizar la opción 6 para visualizar las estadísticas finales, incluyendo Speedup y Eficiencia.
11. Seleccionar la opción 7 para finalizar la ejecución del programa.

10.2 Capturas adicionales

```
=====
SISTEMA DETECTOR DE FRAUDE BANCARIO
=====
1. Generar archivo CSV
2. Leer archivo CSV
3. Ejecutar versión secuencial
4. Ejecutar versión paralela
5. Comparar resultados
6. Mostrar estadísticas
7. Salir
=====
Seleccione una opción: 1
¿Cuántas transacciones desea generar? 500000
¿Cuántas transacciones fraudulentas desea generar? 5000
=====
Archivo generado correctamente.
Ruta: transacciones_20260807_212527.csv
Total de transacciones: 500,000
Normales: 495,000
Fraudulentas: 5,000
=====
Presione una tecla para continuar...
|
```

 transacciones_20260807_212527

7/8/2026 09:25

Archivo de valores se...

72,657 KB

```
=====
SISTEMA DETECTOR DE FRAUDE BANCARIO
=====
1. Generar archivo CSV
2. Leer archivo CSV
3. Ejecutar versión secuencial
4. Ejecutar versión paralela
5. Comparar resultados
6. Mostrar estadísticas
7. Salir
=====
Seleccione una opción: 2
Ingrese la ruta del archivo CSV: C:\P2\FraudShield\src\FraudShield\FraudShield\bin\Debug\net10.0\transacciones_20260807_212527.csv
Se leyeron 500000 transacciones.
Presione una tecla para continuar...
```



```

=====
SISTEMA DETECTOR DE FRAUDE BANCARIO
=====
1. Generar archivo CSV
2. Leer archivo CSV
3. Ejecutar versión secuencial
4. Ejecutar versión paralela
5. Comparar resultados
6. Mostrar estadísticas
7. Salir
=====
Seleccione una opción: 3

===== DETECTOR SECUENCIAL =====
Hilo principal de inicio: 2

Transacciones analizadas: 500000
Resultados generados: 2500000
Tiempo de ejecución: 9423 ms

===== RESUMEN DE FRAUDE =====
Transacciones analizadas : 500000
Aprobadas                : 496637
Canceladas               : 3363
Porcentaje de fraude     : 0.67%

===== TOP DE ALERTAS =====
Monto elevado            : 4570
Comercio riesgoso       : 3693
Horario sospechoso      : 1719
País inusual            : 1323
Alta frecuencia         : 1151

Hilo principal de finalización: 2

Presione una tecla para continuar...

```

```

=====
Seleccione una opción: 4

===== DETECTOR PARALELO =====
Hilo principal de inicio: 2

Procesadores disponibles: 8
Ingrese la cantidad de núcleos a utilizar: 8

Núcleos utilizados: 8
Transacciones analizadas: 500000
Resultados generados: 2500000
Tiempo de ejecución: 2840 ms

===== RESUMEN DE FRAUDE =====
Transacciones analizadas : 500000
Aprobadas                : 496637
Canceladas               : 3363
Porcentaje de fraude     : 0.67%

===== TOP DE ALERTAS =====
Monto elevado            : 4570
Comercio riesgoso       : 3693
Horario sospechoso      : 1719
País inusual            : 1323
Alta frecuencia         : 1151

Hilo principal de finalización: 2

Presione una tecla para continuar...

```

```

=====
SISTEMA DETECTOR DE FRAUDE BANCARIO
=====
1. Generar archivo CSV
2. Leer archivo CSV
3. Ejecutar versión secuencial
4. Ejecutar versión paralela
5. Comparar resultados
6. Mostrar estadísticas
7. Salir
=====
Seleccione una opción: 5

===== COMPARACION =====

Tiempo secuencial : 9423.00 ms
Tiempo paralelo   : 2840.00 ms
Speedup           : 3.32x

La versión paralela fue más rápida.

Presione una tecla para continuar...

```

```

=====
SISTEMA DETECTOR DE FRAUDE BANCARIO
=====
1. Generar archivo CSV
2. Leer archivo CSV
3. Ejecutar versión secuencial
4. Ejecutar versión paralela
5. Comparar resultados
6. Mostrar estadísticas
7. Salir
=====
Seleccione una opción: 6

===== ESTADISTICAS =====

Procesadores disponibles : 8
Núcleos utilizados       : 8

Tiempo secuencial        : 9423.00 ms
Tiempo paralelo          : 2840.00 ms

Speedup                  : 3.32x
Eficiencia                : 41.47%

Presione una tecla para continuar...

```

```

=====
Seleccione una opción: 7

C:\P2\FraudShield\src\FraudShield\FraudShield\bin\Debug\net10.0\FraudShield.exe (proceso 5172) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas ->Opciones ->Depuración ->Cerrar
detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .

```

10.3 Repositorio del proyecto

El código fuente del proyecto se encuentra disponible en el siguiente repositorio de GitHub:

<https://github.com/fairybugs/FraudShield>